STUDENT LEARNING MATERIAL

Cryptography Fundamentals

Keeping secrets, proving truth, and trusting data — from the basics to today's frontier.

Week 7 · Lecture Module
College of Computer Studies
University of Perpetual Help System DALTA

How to use this material

This module is written so you can study it on your own, before or after the lecture. Read it in order — each idea builds on the last. Key terms are **highlighted** the first time they appear. Work through the review questions at the end to check your understanding.

Learning outcomes

By the end of this module, you should be able to:

- **Explain** the core goals of cryptography and how they support the CIA Triad.
- **Differentiate** between symmetric encryption, asymmetric encryption, and hashing, and say when each is used.
- **Describe** how passwords are stored safely using salting and slow hashing.
- **Recognize** modern cryptographic technologies — AEAD, TLS 1.3, and post-quantum standards.

Contents

1 Why Cryptography Matters

2 The Language of Cryptography

3 The Three Cryptographic Tools

4 Storing Passwords Safely: From Hashing to bcrypt

5 A Hands-On Look: The Caesar Cipher

6 Cryptography in the Real World Today

7 The Quantum Threat and Post-Quantum Cryptography

8 Summary and Review Questions

1 Why Cryptography Matters

Every security control you have met so far quietly depends on one thing: mathematics that can scramble data, verify it, and prove who sent it. That mathematics is **cryptography**.

Think back to the CIA Triad — Confidentiality, Integrity, and Availability. Without cryptography, those goals are only wishes. You cannot keep data *confidential* if you cannot scramble it so others cannot read it. You cannot trust a file's *integrity* if you cannot detect when a single character has been changed. And you cannot be sure *who* you are really talking to without a way to prove identity. Cryptography is the machinery that turns these goals from aspirations into something you can actually enforce.

Cryptography does three distinct jobs, and it helps to keep them separate in your mind from the start:

Job	What it provides	In plain terms
Confidentiality	Encryption	Scrambles data so that only the right person, holding the right key, can read it.
Integrity	Hashing	Proves that data has not been altered, even by a single character.
Authentication	Digital signatures	Proves who really sent a message, and that it is genuine.

> **WORTH REMEMBERING**
> Cryptography is not one tool but a small toolbox. Most real systems use all three jobs at once — the trick is knowing which tool does which job.

2 The Language of Cryptography

Before going further, you need four words. Get these straight and the rest of the topic falls into place.

Plaintext	The original, readable message — the grades file, the email, the password. Anything before it has been protected.
Ciphertext	The scrambled version. To anyone without the key, it looks like meaningless nonsense.
Key	The secret that locks or unlocks the data. Security depends on protecting the key — not on hiding the method.
Cipher	The method, or recipe, for turning plaintext into ciphertext and back again.

A principle that surprises people

Here is something that catches many students off guard: in good cryptography, **the method is public and only the key is secret**. This is known as **Kerckhoffs's principle**. A strong cipher stays secure even when everyone knows exactly how it works — all the secrecy lives in the key. That is precisely why the world's most trusted algorithms are published openly and studied by researchers everywhere.

> **THE TRAP TO AVOID**

> **Security through obscurity** — inventing your own secret cipher and hoping nobody figures it out — almost always fails. Home-made cryptography has never survived the years of public attack that real algorithms have. The rule professionals live by: **never roll your own crypto.** Use the algorithms the world already trusts.

3 The Three Cryptographic Tools

Cryptography gives you three core tools. They are not interchangeable — each answers a different security question.

3.1 Symmetric Encryption — one shared key

In **symmetric encryption**, the same secret key both locks and unlocks the data. The sender and receiver must agree on that key ahead of time. Whoever holds it can encrypt and decrypt; whoever lacks it sees only ciphertext.

Its great strength is **speed**. Symmetric algorithms are fast and efficient, which makes them ideal for encrypting large amounts of data — whole files, disks, or a stream of network traffic. The modern standard is **AES** (Advanced Encryption Standard), the same idea you used in the CIA Triad lab when you locked a file with a password.

But symmetric encryption has one hard problem: **how do the two sides share the key safely in the first place?** If an attacker intercepts the key while it is being exchanged, the whole scheme falls apart. This single question is what the next tool was invented to solve.

3.2 Asymmetric Encryption — a public and a private key

Asymmetric encryption uses a matched *pair* of keys instead of one shared secret. What one key locks, only its partner can unlock.

> **PUBLIC KEY — SHARE FREELY**
> Anyone may use it to encrypt a message to you, or to check your signature. You can publish it openly, like the slot on a mailbox.

> **PRIVATE KEY — NEVER SHARED**
> Only you hold it. It decrypts messages sent to your public key and creates signatures that only you could have made.

The cleverness is this: because the public key only needs to be *shared*, not *protected*, you can hand it to the whole world and **no secret ever travels**. That solves symmetric's key-sharing problem completely. Asymmetric encryption (using algorithms like **RSA** and **ECC**) is what secures the little padlock in your browser and lets it set up a secure connection with a website it has never met before.

3.3 Choosing between them — in practice, both

You rarely pick just one. Real systems combine them, each covering the other's weakness: asymmetric encryption safely exchanges a key, then fast symmetric encryption protects the actual data.

	Symmetric	Asymmetric
Keys	One shared secret key	A public + private pair
Speed	Very fast — good for large data	Slower — good for small data
Key sharing	The hard part	Solved by design
Typical use	Encrypting the actual data	Exchanging the symmetric key; signatures

3.4 Hashing — a one-way fingerprint

The third tool is different from the first two. **Hashing** is **not encryption** — there is no key and no way to reverse it. A hash function takes any input and produces a fixed-length *fingerprint* of it. It does not hide data; it **proves integrity**.

Hashing has four properties worth memorizing:

- It turns any input into a fixed-length fingerprint, no matter how large the input.
- The same input always produces the same hash.
- Change even one character, and the hash changes completely.
- You cannot work backwards from the hash to the original data.

That third property is what makes hashing so useful for integrity. You saw it in the Integrity lab, where a keyed hash caught a tampered grades file. Here is the effect in miniature:

```
# SHA-256 — one character changes everything
"grade: 88"  →  a5f3...9c2e
"grade: 89"  →  7b10...44af
```

> **COMMON USES**
> Verifying downloaded files (checksums), detecting tampering, and — as the next section explains — storing passwords safely. Common hash functions today are **SHA-256** and **SHA-3**. The older MD5 and SHA-1 are broken and must not be used for security.

---

4 Storing Passwords Safely: From Hashing to bcrypt

Hashing has one everyday job worth studying closely, because it is where beginners most often go wrong: storing passwords.

The right idea is simple: **never store the password itself**. Instead, store its hash, and at login, hash whatever the user typed and compare the two hashes. If they match, the password was correct — and the real password was never kept anywhere. So far, so good.

4.1 Why a plain hash is not enough

The problem is that a plain, fast hash on its own does not stop a determined attacker who steals the hash file. Three weaknesses combine:

- **Fast is bad here.** SHA-256 is designed to be fast. A modern graphics card can try *billions* of password guesses per second against a stolen hash.

- **Same password, same hash.** Plain hashing gives identical passwords the identical hash — a pattern an attacker can spot instantly, revealing which users share a password.
- **Rainbow tables.** Attackers keep giant precomputed tables mapping common hashes back to their passwords. A bare hash is often just a table lookup away from being cracked.

4.2 Defense one: salting

A **salt** is random data mixed into each password before it is hashed. Every password gets its own unique salt. The effect is powerful:

- Identical passwords now produce completely different hashes.
- Precomputed rainbow tables become useless — they would need to be rebuilt for every possible salt.
- Attackers must crack each password individually instead of all at once.

```
# the same password, two different salts
"P@ss1" + salt_A  →  9f2a...c1
"P@ss1" + salt_B  →  3d7e...88
```

> **A COMMON MYTH**
> Salts do **not** need to be secret. They are stored right next to the hash. Their job is to be *unique*, not hidden — the password is still the only real secret.

4.3 Defense two: slow, memory-hard hashing

Salting stops the shortcuts, but the real defense is to make each guess **expensive**. This is the job of algorithms built to be deliberately slow:

Algorithm	Status	What to know
Argon2id	The current top pick	Winner of the Password Hashing Competition and today's OWASP-recommended default. It is *memory-hard*, which resists cracking by graphics cards and custom chips.
bcrypt	Still safe, widely used	Based on the Blowfish cipher. A tunable *work factor* sets how slow it is — raise it as hardware speeds up. A work factor of 10 or more is recommended.
PBKDF2 / scrypt	The alternatives	PBKDF2 is the FIPS-approved option (use a high iteration count). scrypt is memory-hard like Argon2. Good choices when the top pick is unavailable.

> **THE KEY IDEA**
> A delay you never notice at login — a fraction of a second — becomes *years* of computing time for an attacker trying to guess billions of passwords. That asymmetry is the whole point.

4.4 Putting it together — how passwords should be stored today

Combine the pieces, and a stolen password file becomes almost useless to an attacker:

1. **Take the password** — held only for a moment in memory.

2. **Add a unique salt** — freshly generated for this user, stored beside the hash.
3. **Run a slow hash** — feed password + salt into Argon2id or bcrypt with a sensible work factor.
4. **Store the result** — save the salt and the slow hash, never the password. Check the same way at login.

> **CONNECTS TO YOUR LABS**
> You saw the attacker's side of this when you cracked weak hashes and then re-protected them with salting and a slow hash. This is the defense that makes that cracking fail.

5 A Hands-On Look: The Caesar Cipher

The oldest trick in the book is a good way to *see* plaintext, key, and ciphertext working together. In a **Caesar cipher**, you shift each letter forward by a fixed number — the key. With a shift of 3, A becomes D, B becomes E, and so on.

```
Plaintext:   H E L L O
Key:         shift by 3 →
Ciphertext:  K H O O R
```

> **WHAT IT SHOWS WELL**
> The same key both scrambles and unscrambles the message — this is symmetric encryption in its simplest form. The key (the shift amount) is the whole secret.

> **WHY IT IS NOT ENOUGH TODAY**
> There are only 25 possible shifts, so a computer breaks it instantly, and letter-frequency patterns give it away. Real ciphers like AES use enormous keys to make guessing hopeless.

6 Cryptography in the Real World Today

You have met the three tools. Here are the specific technologies that put them to work in the systems you use every day.

6.1 The algorithms behind the padlock

Algorithm	Type	Role
AES-256 (GCM)	Symmetric	The workhorse. Encrypts most of the world's data at rest and in transit. GCM mode also checks integrity as it encrypts.
ChaCha20-Poly1305	Symmetric	A fast alternative to AES, often preferred on phones and chips without AES acceleration.
RSA & ECC (X25519)	Asymmetric	ECC achieves the same security as RSA with much smaller keys. X25519 now powers most secure key exchange on the web.
SHA-256 / SHA-3	Hashing	The current standard fingerprints. (SHA-1 and MD5 are broken and retired.)

6.2 AEAD: encrypt and verify in one step

Older systems encrypted data with one algorithm, then bolted on a separate integrity check — two steps that were easy to get wrong. Modern ciphers use **AEAD** (Authenticated Encryption with Associated Data), which does both jobs in a single, tested operation: it hides the data *and* guarantees it was not tampered with. Both AES-GCM and ChaCha20-Poly1305 are AEAD ciphers. Combining the two jobs removes a whole class of mistakes — which is why the modern web protocol, described next, allows only AEAD ciphers.

6.3 TLS 1.3 — all three tools working together

Every time you load a secure website (the **https://** padlock), a handshake happens in milliseconds that uses all three cryptographic tools at once:

1. **Verify identity** — the server proves who it is with a certificate and a digital signature (asymmetric).
2. **Exchange a key** — both sides agree on a shared secret using temporary keys, so no secret is ever sent (asymmetric key exchange).
3. **Encrypt the traffic** — the rest of the session is protected with a fast AEAD cipher (symmetric).

> **FORWARD SECRECY**
> Because each session uses fresh, temporary keys, stealing the server's private key *later* cannot decrypt traffic recorded earlier. TLS 1.3 makes this mandatory, and today it carries the majority of web traffic.

---

7 The Quantum Threat and Post-Quantum Cryptography

Cryptography is not finished evolving. A new kind of computer is changing what "secure" means.

A sufficiently large **quantum computer** could break RSA and ECC — the asymmetric cryptography securing almost all of today's internet — because it could solve the underlying math problems far faster than any normal computer. Not everything is equally at risk:

> **WHAT BREAKS**
> Asymmetric algorithms (RSA, ECC). Key exchange and digital signatures are the most exposed.

> **WHAT SURVIVES**
> Symmetric encryption (AES) and hashing (SHA-256) are far more resistant — mostly they just need larger keys.

> **HARVEST NOW, DECRYPT LATER**
> Attackers can record encrypted data *today* and simply wait for quantum computers to catch up. That makes protecting sensitive, long-lived data a problem for the present, not just the future.

7.1 The response: post-quantum standards

In 2024, the U.S. National Institute of Standards and Technology (NIST) finalized the first **post-quantum cryptography** standards — algorithms designed to resist quantum attacks. They are ready to use now:

Standard	Reference	Role	Notes
ML-KEM	FIPS 203	Key exchange	Replaces RSA/ECC for sharing keys. Built on hard lattice-math problems.
ML-DSA	FIPS 204	Digital signatures	The primary new standard for signing and proving identity.
SLH-DSA	FIPS 205	Backup signatures	Hash-based, kept as a safety net relying on different math.

Organizations worldwide have already begun migrating, with deadlines around 2030 to switch critical systems.

7.2 The frontier: cryptography that does more than hide data

Newer techniques let us compute on, and prove things about, data we never actually see:

- **Homomorphic encryption** — compute directly on encrypted data without decrypting it, so a cloud service can process your data while never seeing it.
- **Zero-knowledge proofs** — prove a statement is true without revealing why (for example, prove you are over 18 without showing your birthdate).
- **End-to-end encryption** — only the sender and receiver hold the keys, so even the service carrying the message cannot read it.
- **Multi-party computation** — several parties jointly compute a result from private inputs without any of them revealing their own data.

8 Summary and Review Questions

Key takeaways

- Cryptography enforces the CIA Triad: it hides data, proves integrity, and confirms identity.
- Security lives in the **key**, never in hiding the method — so never invent your own cipher.
- **Symmetric** = one shared key, fast, for the data itself.
- **Asymmetric** = public/private pair, solves key sharing and enables signatures.
- **Hashing** = one-way fingerprint, for integrity and password storage — not encryption.
- Store passwords with a **unique salt** and a **slow hash** (Argon2id or bcrypt), never a plain fast hash.
- Modern systems use **AEAD** ciphers and **TLS 1.3**; **post-quantum** standards are already arriving.

Review questions

Answer in your own words. Suggested answers are shown in italics — try each question before reading them.

1. Explain Kerckhoffs's principle in one sentence, and say why "security through obscurity" is discouraged.

A cipher should stay secure even if its method is public, because all secrecy rests in the key; obscurity fails because home-made secret methods never survive expert scrutiny.

2. A company needs to encrypt a very large database quickly. Symmetric or asymmetric encryption — and why?

Symmetric, because it is fast and efficient for large amounts of data; asymmetric would be too slow.

3. Why is hashing, not encryption, the right tool for storing passwords?

Hashing is one-way, so the system never has to keep the actual password; it stores only the fingerprint and compares hashes at login.

4. A developer hashes passwords with plain SHA-256 and no salt. Give two reasons this is unsafe.

SHA-256 is fast, so attackers can try billions of guesses per second; and without salts, identical passwords share a hash and rainbow tables can crack them by lookup.

5. What does a salt achieve, and does it need to be kept secret?

It makes identical passwords hash differently and defeats rainbow tables; it needs to be unique, not secret, and is stored next to the hash.

6. Name the OWASP-recommended password hashing algorithm today, and one acceptable alternative.

Argon2id is the recommended default; bcrypt (work factor 10+) or PBKDF2/scrypt are acceptable alternatives.

7. When you load an https:// site, which cryptographic tool verifies the server's identity, and which encrypts the traffic?

A digital signature (asymmetric) verifies identity; a fast AEAD symmetric cipher encrypts the session traffic.

8. Why is "harvest now, decrypt later" a present-day concern rather than only a future one?

Attackers can record today's encrypted data now and decrypt it once quantum computers mature, so long-lived secrets are already at risk.

End of Week 7 module. Prepared for Information Assurance and Security (BSCS 3110), College of Computer Studies, University of Perpetual Help System DALTA.